Relatório do Trabalho Prático I: Pipeline de ETL no Pentaho Kettle para Monitorização de Smart Cities

1. Enquadramento

- **Unidade Curricular:** Integração de Sistemas de Informação (ISI)
- **Curso:** Licenciatura em Engenharia de Sistemas Informáticos (LESI)
- **Instituição:** Instituto Politécnico do Cávado e do Ave (IPCA)
- **Ano Letivo:** 2026/2027
- **Docentes:** Prof. Luís Ferreira & Prof. Óscar Ribeiro
- **Autor:** Lucas Lima Magalhães
- **Repositório Git :**https://github.com/LucasMagalhaes-sys/TP01

2. Problema

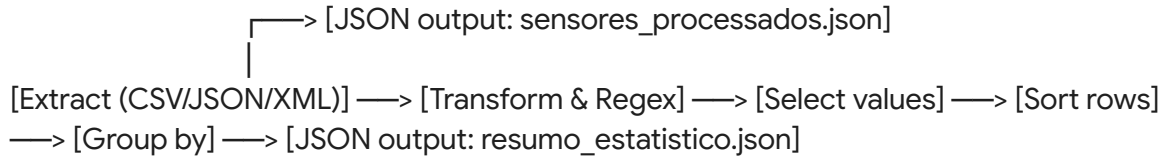
No contexto emergente dos ecossistemas urbanos inteligentes (*Smart Cities*), os municípios e gestores de infraestruturas enfrentam o desafio constante de consolidar dados provenientes de múltiplos sistemas e dispositivos IoT. Neste cenário, a informação encontra-se frequentemente dispersa por fontes heterogéneas, com estruturas incompatíveis, desprovida de validação centralizada e sujeita a erros de formato.

O objetivo deste trabalho prático consiste no desenvolvimento de uma solução automatizada de integração de dados baseada em processos de **ETL (Extract, Transform, Load)** recorrendo ao **Pentaho Data Integration (Kettle)**. A aplicação foca-se na monitorização e manutenção de sensores urbanos (como estações de Qualidade do Ar e sensores de Tráfego), onde é necessário:

1. Ingerir dados de registos técnicos mantidos em ficheiros CSV.
2. Cruzar a informação com metadados geográficos e operacionais armazenados em ficheiros JSON.
3. Aplicar regras de negócio e limites de alerta estipulados em ficheiros XML.
4. Validar os identificadores dos sensores recorrendo a **Expressões Regulares (Regex)**.
5. Garantir a resiliência da operação através de orquestração automatizada e tratamento de exceções.

3. Estratégia Utilizada

A arquitetura da solução foi estruturada em três fases fundamentais (E-T-L) e gerida por um fluxo de orquestração superior (Job):



Camadas de Processamento:

- **Fase 1: Recolha e Preparação de Dados ("E")**

  Ingestão paralela de três origens com formatos distintos: manutencoes.csv (histórico de intervenções), sensores_meta.json (localizações e datas de instalação) e limites_alertas.xml (limiares críticos de alerta por tipo de sensor).
- **Fase 2: Processamento e Transformação ("T")**

  - **Ordenação e Cruzamento:** Utilização dos passos Sort rows e Merge join para combinar os registos do CSV com os metadados do JSON através do campo id_sensor.
  - **Validação por Regex:** Aplicação do operador Regex evaluation ao identificador do sensor com a expressão ^SNSR-\d{3}$ para validar a conformidade da nomenclatura.
  - **Enriquecimento por Busca:** Utilização do Stream lookup para cruzar o tipo de sensor com o ficheiro XML e associar o respetivo limite de alerta (limite_maximo).
  - **Sanitização de Schema:** Utilização do Select values para remoção de colunas redundantes resultantes do *join* (id_sensor_1).
  - **Agregação Estatística:** Ordenação adicional e utilização do Group by para calcular o número total de sensores ativos e a média dos limites de alerta agrupados por categoria (tipo).
- **Fase 3: Persistência e Exportação ("L")**

  Exportação automatizada para a pasta dados_saida/ através de dois ficheiros JSON:

1. sensores_processados.json: Dataset granular unificado e validado.
2. resumo_estatistico.json: Dataset executivo com os indicadores agrupados.

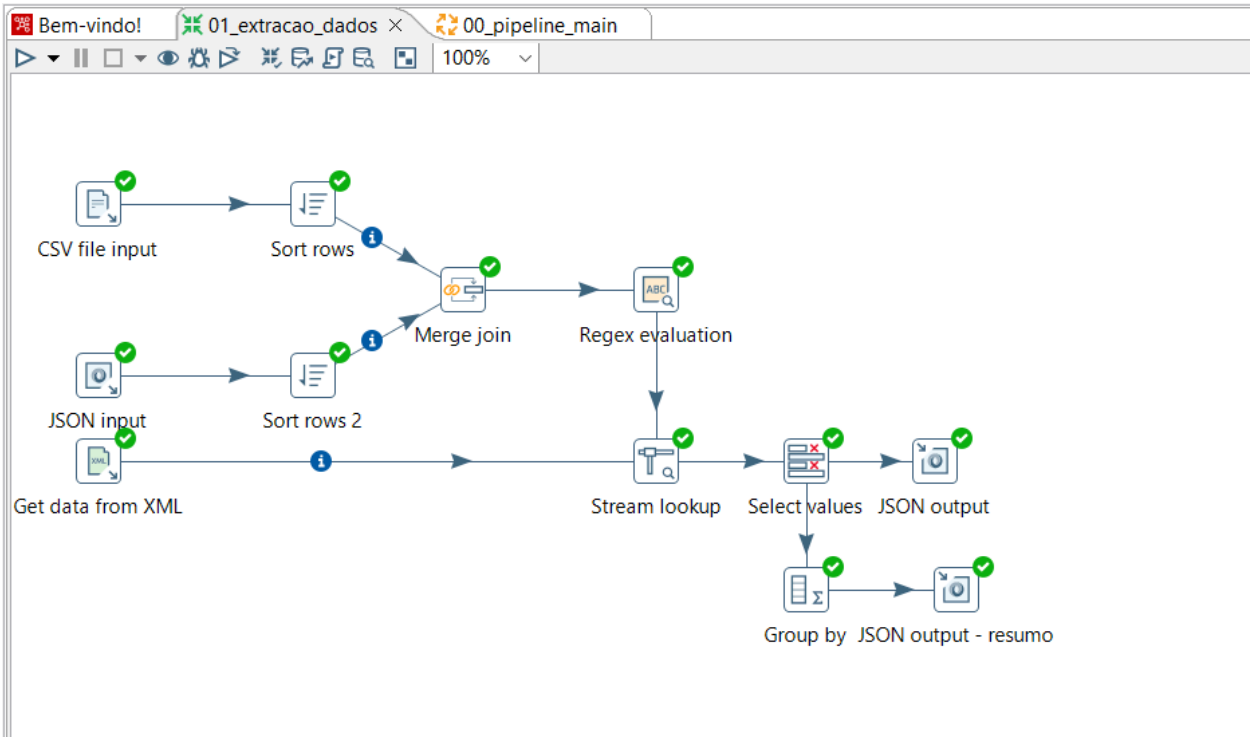
- **Orquestração e Controlo de Processo (Job):**

  Implementação do Job 00_pipeline_main.kjb responsável pela validação da existência do ficheiro de entrada (File exists), desencadeamento da transformação (Transformation),

registo de mensagens de auditoria (Write to log), notificação automatizada em caso de falha (Mail) e encerramento preventivo (Abort job).

-

4. Transformações (01_extracao_dados.ktr)



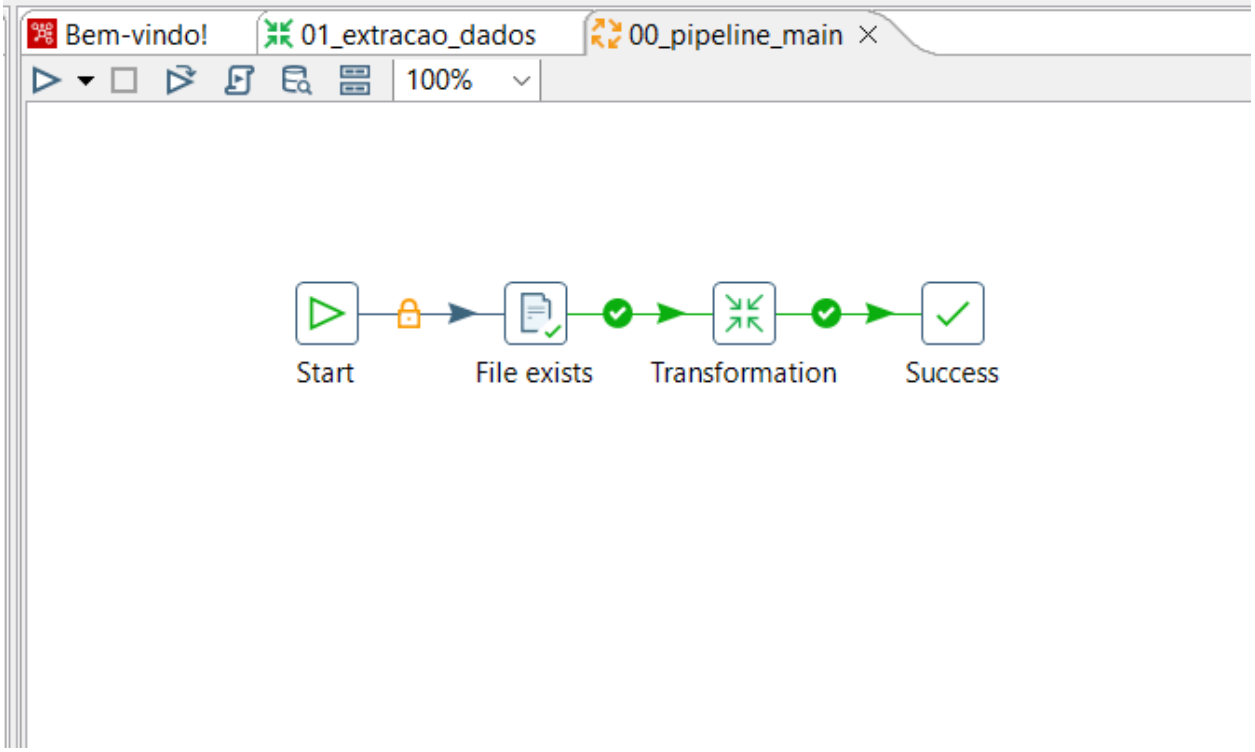
Descrição dos Componentes e Fluxo de Dados:

1. **CSV file input (manutencoes.csv):** Lê os registos de manutenções (id_manutencao, id_sensor, data_revisao, estado, tecnico).
2. **JSON input (sensores_meta.json):** Extrai a localização, tipo de sensor e data de instalação a partir da estrutura JSON.
3. **Sort rows & Sort rows 2:** Ordenam as duas streams pelo campo de chave id_sensor (requisito obrigatório para execução do *Merge join*).
4. **Merge join:** Realiza um *Inner Join* entre a stream do CSV e do JSON com base na correspondência de id_sensor.
5. **Regex evaluation:** Avalia o campo id_sensor com o padrão regular ^SNSR-\d{3}$, criando o campo booleano formato_sensor_valido.
6. **Get data from XML (limites_alertas.xml):** Ingere o ficheiro XML parsing os nós tipo_sensor e limite_maximo.
7. **Stream lookup:** Associa a cada registo o respetivo limite de alerta cruzando o campo tipo com tipo_sensor.
8. **Select values:** Filtra o schema final, renomeia o campo do XML para limite_alerta e remove a coluna duplicada id_sensor_1.
9. **JSON output (sensores_processados.json):** Grava o conjunto final de dados

transformados e validados no disco.

10. **Sort rows (tipo) & Group by:** Agrupa os registos por tipo de sensor e calcula as agregações total_sensores (Number of Values) e media_limite_alerta (Average).
11. **JSON output - Resumo (resumo_estatistico.json):** Exporta o ficheiro de resumo estatístico.

5. Jobs (00_pipeline_main.kjb)



Fluxo de Controlo de Processo (*Process Control*):

O Job 00_pipeline_main.kjb garante que a transformação apenas é executada se os pré-requisitos de sistema forem cumpridos, assegurando a gestão graciosa de falhas:

1. **Start:** Ponto de entrada do pipeline.
2. **File exists:** Verifica a existência física do ficheiro essencial dados_entrada/manutencoes.csv no sistema de ficheiros.
3. **Caminho de Sucesso (Linha Verde):**
   - Se o ficheiro existir, dispara a execução do passo **Transformation** (01_extracao_dados.ktr).
   - Após a conclusão bem-sucedida da transformação, o fluxo avança para **Success**.
4. **Caminho de Exceção / Falha (Linha Vermelha):**
   - Caso o ficheiro de entrada não seja localizado ou ocorra um erro durante a execução da transformação, o fluxo é desviado para o passo **Write to log**, que regista a mensagem de erro detalhada no log de execução.
   - Em seguida, o passo **Mail** dispara uma notificação por correio eletrónico para o administrador do sistema (admin@smartcity.pt) com o assunto [FALHA ETL] Erro no

processamento de sensores.
    - Por fim, o passo **Abort job** interrompe formalmente a execução do processo, prevenindo a proliferação de dados corrompidos.

6. Vídeo com Demonstração



- **Link direto para o vídeo:** https://youtu.be/g8CUr558Yjk

7. Conclusão e Trabalhos Futuros

Conclusão

O trabalho prático atingiu na íntegra os objetivos propostos para a unidade curricular de Integração de Sistemas de Informação. Através da ferramenta Pentaho Data Integration (Kettle), foi demonstrada a capacidade de ingerir e integrar três origens heterogéneas de dados (CSV, JSON e XML), implementar mecanismos rigorosos de validação através de Expressões Regulares, realizar cruzamentos complexos e agregações estatísticas, e orquestrar todo o ciclo de vida com mecanismos automatizados de gestão de erro.

Trabalhos Futuros

Como evolução da arquitetura desenvolvida, propõem-se as seguintes melhorias:

1. **Ingestão em Tempo Real (Streaming):** Integração com um *broker* MQTT (ex.: Mosquitto) e workflows do **Node-RED** para capturar leituras contínuas dos sensores em tempo real antes do processamento.
2. **Persistência em Base de Dados Relacional:** Carregamento dos dados limpos para tabelas num SGBD PostgreSQL / PostGIS para permitir consultas espaciais.

3. **Visualização Dinâmica:** Alimentação de um *dashboard* executivo no **Grafana** ou **Power BI** a partir dos ficheiros JSON gerados, permitindo aos gestores urbanos a monitorização geográfica e temporal dos alertas de manutenção em tempo real.

8. Referências Bibliográficas

1. **Ferreira, L., & Ribeiro, Ó.** (2026). *Enunciado do Trabalho Prático I — Integração de Sistemas de Informação*. Licenciatura em Engenharia de Sistemas Informáticos, Instituto Politécnico do Cávado e do Ave (IPCA).
2. **Hitachi Vantara.** (2024). *Pentaho Data Integration (Spoon) Documentation*. Disponível em: https://help.hitachi-vantara.com/.
3. **Friedl, J. E. F.** (2006). *Mastering Regular Expressions* (3rd ed.). O'Reilly Media.